DAT158 End-to-end ML project

Thobias Kvalvik Høivik

1 The idea

My idea for the project was to make a (very simplified) physical simulation where you have a spacecraft, unaffected by gravity, whose goal is to move somewhere. Crucially, the only things the spacecraft should be allowed to do are rotate, and choose how much thrust is applied.

This idea was inspired by this old video by Pezza's Work which I have always wanted to try to recreate.

Now, the most natural approach, which is the same used by said creator, is to use a small neural network, which to my understanding lies a bit outside the scope of the syllabus, but as the solution I landed on is a form of reinforcement learning I figured it'd be fine.

The demo may be found on my personal website.

I won't devote an entire section to the simulation, but it is quite simple. The controller chooses thrust and torque and the simulation updates how those change the motion of the rocket. The rocket has a position, velocity, rotation, and angular velocity. They are updated each $\Delta t = 1/30$ seconds. Thrust adds acceleration of up to 4 m/s^2 along the nose, and torque adds angular acceleration of up to 6 rad/s^2. Each step also multiplies linear velocity by 0.99 and angular velocity by 0.94. This artificial damping makes both motion and spinning gradually slow down.

2 How does it work?

In supervised learning, we might have an input x, a known answer y, and a prediction $\widehat{y}$ with a loss such as $(\widehat{y} - y)^2$ which tells you how to adjust the model.

In our problem, where we are simulating a rocket in space, we want the model to have access to the current rocket's situation with the model then choosing thrust and torque.

We do not have a predetermined answer to what the correct answer should be. We may only run the simulation and judge what happens next. This also creates a slight complication, that being that an action changes what situation the model will encounter next.

The **policy** is the model's rule for choosing actions from observations. This implementation uses two networks,

- An actor, who chooses how to control the rocket, and
- A critic, who estimates how much future reward the current situation is likely to produce.

The critic's job is to help train the actor, and does not issue any steering commands.

In each simulation step, the actor is given seven numbers

$$s_t = \begin{bmatrix} (g_x - x)/3 \\ (g_y - y)/3 \\ v_x/3 \\ v_y/3 \\ \sin\theta \\ \cos\theta \\ \omega/6 \end{bmatrix}$$

where the first two entries correspond to the signed horizontal and vertical offsets from the goal, the next two represent velocity at this moment, those after that correspond to rotation and the last is the angular velocity (how much the rocket is spinning).

Positions and velocities are scaled to keep their ranges manageable and heading/rotation uses sine and cosine since we don't want to deal with $0 \equiv 2\pi$ wrap-around shenanigans. All entries except sine and cosine are also clipped to $[-2, 2]$. The network does not receive the target's velocity.

The actor is then a small network with 7 inputs, two hidden layers with 32 neurons each, and 2 outputs. This is pretty much identical to the architecture used in the aforementioned YouTube video, except that there the two hidden layers have only 9 neurons.

Each hidden neuron in our network computes a weighted sum, adds a bias and applies tanh which lets the network represent nonlinear relationships so the appropriate thrust can depend jointly on a bunch of observations in a potentially weird way.

It is worth mentioning that during training, the actor describes a distribution of possible actions and its two outputs are means, written μ, for two Gaussian distributions. We sample them such:

$$u = \mu + \sigma\epsilon, \ \epsilon \sim \mathcal{N}(0, 1)$$

where

- μ is the network's preferred output,
- σ controls the amount of variation, and
- ϵ supplies some new random variation.

The implementation also learns the two standard deviations, which are shared across observations, whereas the means change depending on what is required given the rocket's current state.

Sampling provides a sort of exploration. If the model always repeated its current preferred action it would collect little evidence about alternatives. A stochastic policy tries "nearby" alternatives and can potentially discover some which work better.

This approach is described in Spinning Up's Introduction.

The sampled numbers u are latent outputs and they can, annoyingly, be arbitrarily large so we convert them into bounded controls,

$$T = \frac{\tanh(u_0) + 1}{2}, \ \tau = \tanh(u_1).$$

This gives thrust T between 0 and 1 and torque τ between -1 and 1.

After applying these controls, the simulation produces a reward. The scoring rule rewards approaching and remaining near the target, and penalizes distance, excessive speed near the target, "fuel use", torque and spinning.

More precisely, writing d_t and d_{t+1} for the distances before and after a physics step with the same target held fixed, the reward is

$$\begin{aligned} q_t &= \exp(-2d_{t+1}^2), \\ r_t &= 4(d_t - d_{t+1}) - 0.10d_{t+1} + 0.15q_t \\ &\quad - 0.08q_t\|\mathbf{v}_{t+1}\|^2 - 0.008T_t \\ &\quad - 0.004\tau_t^2 - 0.002\omega_{t+1}^2. \end{aligned}$$

An escape adds a penalty of -1. "Fuel use" is an engine-use penalty, rather than an actual tank that empties.

This works reasonably well, but we will discuss some annoying side effects from this metric later on.

Immediate reward alone is not sufficient. If the rocket approaches the target quickly, braking might briefly increase its distance from the target, but prevent a large overshoot. An algorithm that judged only the next instance could discourage useful braking.

We therefore care about return, i.e. accumulated future reward. The implementation multiplies the contribution by 0.99 for each additional step,

$$G_t = r_t + 0.99r_{t+1} + 0.99^2 r_{t+2} + \cdots$$

Here r_t is the immediate reward and G_t represents longer-term outcome.

The critic's job, as mentioned, is to predict the longer-term outcome. We denote the prediction $V(s)$, where s is the observation. For instance, it might learn that "close to the target, but quickly moving away" is less promising than "slightly farther away but moving slowly toward it".

This leads to an advantage, i.e. an estimate of how favorable the sampled decision was relative to what the critic expected.

The first term is a temporal-difference error,

$$\delta_t = r_t + 0.99V(s_{t+1}) - V(s_t),$$

which we can read as the reward received, in addition to the estimated value of the next state, minus the value we previously expected.

The implementation combines error from successive steps using Generalized Advantage Estimation,

$$A_t = \delta_t + (0.99 \cdot 0.95)A_{t+1}.$$

Repeatedly expanding this expression gives the current error plus progressively smaller contributions from later errors. It lets later consequences influence the assessment of an earlier action. The 0.95 parameter balances reliance on short-term critic predictions against reliance on longer, noisier sequences of experience. That is the purpose of GAE.

These equations describe an uninterrupted trajectory. An escape removes the next-state value term. A time limit retains that term using the final observation, but the advantage recursion stops across either kind of episode reset.

These are raw advantages. Before updating the actor, they are centered using the whole batch's mean and divided by its standard deviation (with a small numerical safeguard). I will write the normalized advantages as $\widetilde{A}_t$. Centering can change an individual sign: a positive normalized advantage means favorable relative to the batch average. The critic still uses the raw advantages.

Now we can elucidate how the critic helps guide the actor.

We save the action the actor actually sampled. During optimization, we ask how likely the current actor considers the same action,

- a favourable normalized advantage encourages increasing its likelihood, while
- an unfavourable normalized advantage encourages decreasing its likelihood.

Since the outputs are continuous (as a theoretical compsci person I would say not really, but it's a fine assumption), likelihood means probability density. We make actions near the sampled value more or less likely.

The basic policy-gradient learning signal involves

$$\widetilde{A}_t \log \pi_w(u_t \mid s_t),$$

where π_w is the action distribution produced by actor weights w. The density is already differentiable; its logarithm gives a useful gradient for changing the relative likelihood of sampled outputs. Backpropagation computes how changing each weight affects this expression, and the optimizer can update the weights accordingly.

We do not backpropagate through the rocket's physics; instead the simulator supplies experience, while the gradient is computed through the actor's likelihood calculation. The policy-gradient derivation explains why this can estimate how to improve expected return.

For one Gaussian output, the derivative of log-likelihood with respect to its mean is

$$\frac{\partial \log \pi(u)}{\partial \mu} = \frac{u - \mu}{\sigma^2}.$$

If the sampled output was above the mean, a positive normalized advantage tends to pull the mean upward towards it, while a negative normalized advantage tends to push the mean further away. The full network update combines many such samples, so an individual sample does not determine the result by itself.

The algorithm used, Proximal Policy Optimization, or PPO, adds a restraint. We reuse a batch for several passes, but that batch comes from the actor before the updates. Large changes would make the collected experience less representative of the current policy.

PPO compares the new and old densities for each recorded action,

$$\rho_t = \frac{\pi_{\text{new}}(u_t \mid s_t)}{\pi_{\text{old}}(u_t \mid s_t)}.$$

A ratio of 1.1 means its density increased by 10%. With the clipping setting used,

- for positive normalized advantage, this sample stops providing further encouragement once its ratio exceeds 1.2, while
- for negative normalized advantage, it stops providing further encouragement to decrease its density once the ratio falls below 0.8.

This removes the incentive for further favourable movement from that sample's part of the objective. Other samples can still change the same weight. This behavior is explained in the PPO documentation.

We then fit the critic with squared error against the targets

$$Y_t = V_{\text{old}}(s_t) + A_t,$$

using the raw advantages. These targets are generated from rewards and existing value predictions, and are kept fixed during the five optimization passes.

The training loop in the training script is

1. Run 128 rocket simulations, collecting 128 steps for each, giving $16,384$ transitions.
2. Save observations, sampled outputs, rewards, old action densities, and critic predictions.
3. Calculate raw advantages and critic targets, then normalize the advantages.
4. Shuffle the batch and make five optimization passes, using groups of $1,024$ transitions.
5. Update the actor with PPO and the critic with regression.
6. Collect new experience with the updated actor and repeat.

3 Actual implementation and such

The network's derivatives and optimizer were implemented directly with NumPy. The optimizer, Adam, uses recent gradient history to adjust parameter updates and a small entropy bonus encourages the latent action distributions to retain some variation.

Training experience and settings There is no downloaded dataset of correct steering commands. The simulation generates observations, sampled actions, rewards and following observations. Episodes last 240 steps, or 8 seconds, unless the rocket escapes beyond $|x| > 5$ or $|y| > 5$. Starting positions, velocities, headings and targets are randomized. Targets are drawn from a mixture of static positions and smooth sinusoidal paths. During the first 20% of training, the reset conditions and target motion gradually become more difficult. The handwritten controller supplies no training examples.

The final run collected $30,015,488$ transitions with seed 17. The initial actor learning rate was 0.0004, linearly decreasing to about 30% of that rate; the critic used twice the actor rate. PPO clipping was 0.2, and the latent Gaussian entropy coefficient was 0.003. These

settings and the earlier batch sizes are recorded alongside the weights in `public/models/rocket-rl.json`.

Finally, the browser runs a saved actor. At each step it reads the seven inputs, evaluates the network, and converts its means into thrust and torque. It turns off the training-time random sampling. It does not learn anything when interacted with in the page, and it does not search through possible future trajectories.

The network was trained in about 4 minutes on my own desktop computer with specs:

- AMD Ryzen 7 7800X3D,
- 64gb of 5600MHz DDR5,
- NVIDIA GeForce RTX 4080 SUPER (not really relevant).

The measured runtime was 234.27 seconds on the CPU; the GPU was not used.

There is also a handwritten controller explicitly containing rules for pulling toward the target, damping velocity, and turning toward the desired acceleration.

Evaluation Each controller was evaluated on the same 100 seeded episodes of 240 steps, using seed 90210 and randomized static or sinusoidal targets. These scenarios were inspected during development, so the results are validation results rather than an untouched final test.

Success means a mean distance below 0.45 m and mean speed below 0.8 m/s during the final 60 steps. After an escape, remaining steps count as $\sqrt{50}$ m of tracking error, a reward of $-0.1\sqrt{50}$ per step, and zero engine use and spin. This keeps early failure from artificially improving the distance metric; low average thrust alone does not mean successful flight.

Controller	Distance (m)	Thrust	Spin (rad/s)	Return	Success
Random	5.548	0.166	0.087	-151.327	0%
Untrained	5.566	0.165	0.017	-151.674	0%
Early PPO	0.609	0.141	2.669	9.958	96%
Middle PPO	0.397	0.196	2.989	17.947	100%
Final PPO	0.400	0.167	2.349	19.715	100%
Handwritten	0.433	0.075	1.215	21.128	99%

Table 1: Mean validation results. Spin is absolute angular speed; thrust is on a scale from 0 to 1.

Checkpoints were saved at 0, $3,014,656$, $12,009,472$ and $30,015,488$ transitions. The learning curve shown on the lab page uses separate 32-episode evaluations during training.

Reproducibility The root `README.md` describes how to install dependencies, train and evaluate the controller, and build and run the website. The training scripts and their detailed documentation are under `scripts/rocket-rl`. Training requires Python 3.11 or newer and NumPy 2.3.5. From the repository root:

```
python -m pip install -r scripts/rocket-rl/requirements.txt
python scripts/rocket-rl/train.py --steps 30000000 --seed 17
python scripts/rocket-rl/evaluate.py --verify
```

The website requires Node.js 22.12 or newer. Its build and deployment instructions, saved model and simulation source are included in the repository. The implementation was also checked with finite-difference gradients and browser/Python parity tests.

Since the project was developed first and foremost with the goal of putting it on my website, the mangled remains of the project I have put here may be impractical to work with. Sorry for this.

4 Discussion

The end result is a pretty neat toy to play around with and it performs pretty well, being able to reach the target quite effectively, supposing you are using the final policy.

Perhaps unsurprisingly, the hand-written approach looks a lot smoother. The learned controller has slightly lower mean tracking distance, but the handwritten controller uses less thrust, spins less and actually receives higher mean reward. So the learned policy is not simply better according to the reward metric. This points to our maybe not having fine-tuned the reward metric well enough for the kind of flight I wanted. It could also reflect PPO not fully optimizing the reward that was already defined.

A major issue with the model is that if you very rapidly move the target around it is very easy for the rocket to end up flying too far off the screen as the model wants to correct by doing a really wide turn, which isn't great. In fact, putting the goal in the bottom right or top right makes the model send the rocket off the screen every time with the latest policy.

These are observations from interacting with the cursor, rather than results from the fixed validation scenarios above. The target positions and sudden movements can differ from the training distribution, and the seven inputs do not include target velocity. Also, the reset boundary lies beyond the visible flight area, so leaving the screen is not the same as reaching an episode escape. The single training seed and these failures leave plenty of room for improvement. More varied target paths, stronger penalties for unwanted spinning and evaluation across additional seeds would be sensible next experiments; they have not been carried out here.

AI use declaration OpenAI's ChatGPT was, in part, used to converge on certain design choices, along with technical explanations of how such choices are implemented, as well as parts of actual implementation, debugging, and porting the necessary files over from my website repository where the model was first developed.